

Socket Programming in C

- Socket programming is a method used in network communication that allows data to be sent and received between devices.
- Sockets provide a communication channel between two processes, either on the same machine or different machines connected via a network.
-

How Socket Communication Works

1.
 1. **Server Setup:**
 - Create a socket.
 - Bind the socket to an IP address and port.
 - Listen for incoming connections.
 - Accept client connections.
 2. **Client Setup:**
 - Create a socket.
 - Connect to the server.
 3. **Data Exchange:**
 - Send and receive data between the server and client.

How It Works

1. **Server:**

- Creates a socket with `socket()`.
- Binds to an IP/port with `bind()`.
- Waits for connections using `listen()`.
- Accepts incoming connections with `accept()`.
- Receives and sends data using `recv()` and `send()`.

2. **Client:**

- Creates a socket with `socket()`.
- Connects to the server with `connect()`.
- Sends a message using `send()`.
- Receives a response from the server.

Breakdown of the Code: Server

1. Headers:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
```

- These headers include functions for input/output, string manipulation, memory allocation, system calls (like `read` and `write`), and network communication

2. Socket Creation:

```
sockfd = socket(AF_INET, SOCK_STREAM, 0);
```

1.

- Creates a TCP socket.
- AF_INET specifies IPv4, SOCK_STREAM indicates TCP (connection-oriented).

3. Server Address Structure:

```
server.sin_family = AF_INET;  
server.sin_addr.s_addr = INADDR_ANY;  
server.sin_port = htons(8080);
```

1.

- `sin_family` specifies the address family (IPv4).
- `INADDR_ANY` allows the server to accept connections on any network interface.
- `htons(8080)` converts the port number to network byte order (big-endian).

4. Binding the Socket:

```
bind(sockfd, (struct sockaddr *)&server, sizeof(server));
```

- Associates the socket with the IP address and port.

5. Listening for Connections:

```
listen(sockfd, 5);
```

1.
 - Puts the socket in passive mode to listen for incoming connection requests.
 - 5 specifies the backlog, the number of pending connections allowed.

6. Accepting a Connection:

```
newsock = accept(sockfd, (struct sockaddr *)&client, &len);
```

1.
 - Accepts the first incoming client connection and creates a new socket (`newsock`) for communication.
- 2.

7. Receiving Data:

```
read(newsock, buffer, sizeof(buffer));  
  
printf("Message: %s\n", buffer);
```

1.
 - Reads data from the client and prints the received message.
- 2.

8. Sending a Response:

```
send(newsock, "Hello from server", 17, 0);
```

1.
 - Sends a response back to the client.

9. Closing Sockets:

```
close(newsock);
```

```
close(sockfd);
```

- Closes both the connection socket (`newsock`) and the listening socket (`sockfd`).

Breakdown of the Client Code:

1. Socket Creation:

```
sockfd = socket(AF_INET, SOCK_STREAM, 0);
```

- Creates a TCP socket for the client.

2. Server Address Structure:

```
server.sin_family = AF_INET;  
server.sin_port = htons(8080);  
server.sin_addr.s_addr = inet_addr("127.0.0.1");
```

1.

- The `inet_addr("127.0.0.1")` specifies the localhost (server running on the same machine).
- Change the IP to the server's IP if running on different machines.

3. Connecting to Server:

```
connect(sockfd, (struct sockaddr *)&server, sizeof(server));
```

1.

- Connects to the server. The server must be running before this.

4. Sending Data:

```
send(sockfd, buffer, strlen(buffer), 0);
```

1.

- Sends a message to the server.

2.

5. Receiving Response:

```
read(sockfd, buffer, sizeof(buffer));  
printf("Message from server: %s\n", buffer);
```

1.
 - Reads the server's response and displays it.

6. Closing Socket:

```
close(sockfd) ;
```